FORMATO PARA ELABORACIÓNDE TALLERES ESTUDIANTILES	**Departamento:** DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN - DCCO-SS
	Carrera: INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN **Asignatura:** METOD DE DESARROLLO SOFTWARE **NRC:** 30693 **Nro. Grupo:** 3
	Nombre de estudiante líder: Jaime Cabezas

TEMA

Resolución de preguntas de autoevaluación del capítulo 1 y 2

Índice

TEMA 1

ANTECEDENTES 2

OBJETIVOS 2

Objetivos Específicos: 2

DESARROLLO 3

Preguntas Capítulo 1 — Actividad de Aprendizaje (Pág. 17) 3

Preguntas Capítulo 2 — Actividad de Aprendizaje (Pág. 25) 5

Autoevaluación del Capítulo 2 (Págs. 25–27) 8

CONCLUSIONES 11

RECOMENDACIONES 11

Bibliografía 12

ANTECEDENTES

Se han realizado un total de veinte preguntas como parte de auto aprendizaje del libro Fundamentos de Ingeniería de Software un enfoque práctico, las primeras diez preguntas de la unidad uno y las diez ultimas de la unidad dos.

OBJETIVOS

Objetivo General: Definir los fundamentos teóricos y metodológicos de la ingeniería de software aplicados a la gestión de proyectos de TI.

Objetivos Específicos:

Diferenciar la ingeniería de software de las ciencias de la computación y otras ingenierías tradicionales.

Identificar las fases, retos, costos y paradigmas del ciclo de vida del software.

Establecer los perfiles técnicos, herramientas y métricas de calidad necesarias para la ejecución de un proyecto.

DESARROLLO

Preguntas Capítulo 1 — Actividad de Aprendizaje (Pág. 17)

1. ¿Qué es Ingeniería de Software?

Es la aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento de software. ((Sommerville, 2011). *Software Engineering 9. Pearson)*

2. ¿Cuáles son las actividades fundamentales de la ingeniería de software?

Las cuatro actividades básicas son: especificación del software (definir funcionalidad y restricciones), diseño e implementación (crear el software), validación

(verificar que cumple con los requerimientos) y evolución (modificar el software según cambios del cliente). (Sommerville, 2011). *Software Engineering 9. Pearson*

3. ¿Cuál es la diferencia entre ingeniería de software e ingeniería de sistemas de la información (ITIN)?

La Ingeniería de Sistemas de la Información (ITIN) se enfoca en el análisis, diseño e integración de sistemas de información orientados a la gestión organizacional, priorizando los procesos de negocio y la toma de decisiones. La Ingeniería de Software, en cambio, se centra en la creación sistemática de productos software que satisfagan necesidades técnicas del mundo real, aplicando métodos de desarrollo, pruebas y mantenimiento de software a gran escala. (ACM/IEEE, 2020). Computing Curricula CC2020. Adaptado en: Fundamentos de IS, Cap. 1

4. ¿Qué es software?

Es un conjunto estructurado de programas, procedimientos, documentación y datos relacionados que permiten operar un sistema informático. (Pressman, 2005). *Ingeniería del Software: Un Enfoque Práctico. McGraw-Hill*

5. ¿Cuáles son los atributos del buen software?

Un buen software debe ser mantenible (fácil de modificar ante cambios), confiable (seguro y resiliente ante fallos) y fácil de usar por el usuario final, además de eficiente en el uso de recursos del sistema. (Sommerville, 2011). *Software Engineering 9. Pearson*

6. ¿Cuáles son los principales retos que enfrenta la ingeniería de software?

Los retos principales son la heterogeneidad (integrar sistemas en múltiples plataformas), los tiempos de entrega reducidos (presión por llegar rápido al mercado) y la confianza (garantizar seguridad y privacidad en los sistemas desarrollados). (Sommerville, 2011). *Software Engineering 9. Pearson)*

7. ¿Cuáles son los costos de la ingeniería de software?

Aproximadamente el 60% de los costos corresponde al desarrollo y el 40% a las pruebas. El costo del mantenimiento a largo plazo siempre supera al costo inicial de desarrollo del sistema. (Pressman, 2005). *Ingeniería del Software: Un Enfoque Práctico. McGraw-Hill*

8. ¿Cuáles son los mejores métodos y técnicas de la ingeniería de software?

Se distinguen dos grandes enfoques: los métodos ágiles (como Scrum y Kanban) para entornos dinámicos y desarrollo web, y los métodos formales o plan-driven para sistemas críticos donde la seguridad es fundamental, como en salud o aviación. (Sommerville, 2011). *Software Engineering 9. Pearson*

9. ¿Qué entiende por paradigmas de la ingeniería de software?

Son modelos estructurales que guían las fases y el orden del ciclo de vida del desarrollo de software. Los principales paradigmas son el Modelo en Cascada, el Desarrollo Incremental y el proceso orientado a la reutilización de componentes. (Sommerville, 2011). *Software Engineering 9. Pearson*

10. ¿Cuál es la definición del modelo incremental versus otros paradigmas y diferencias con otras ingenierías?

El modelo incremental vincula especificación, desarrollo y validación, generando versiones sucesivas que añaden funcionalidad progresivamente. A diferencia del modelo en cascada (secuencial y rígido), el incremental es iterativo. A diferencia de la ingeniería civil, el software es intangible, maleable y no sufre desgaste físico. (Sommerville, 2011). *Software Engineering 9. Pearson*

Preguntas Capítulo 2 — Actividad de Aprendizaje (Pág. 25)

1. ¿Qué hay en la caja?

Nro. Grupo: 3

Un producto software es el resultado de un proceso de desarrollo con actividades estandarizadas y estructuradas; 'en la caja' se encuentra el conjunto de programas, estructuras de datos, documentación y todos los artefactos que cumplen los requisitos funcionales y no funcionales del cliente. *(Fundamentos de IS, Cap. 2. Basado en Pressman, R. S. (2005))*

2. ¿Cuál es el trabajo que tenemos que hacer para lograr la meta?

El trabajo consiste en ejecutar el ciclo de vida del desarrollo: levantamiento de requisitos (funcionales y no funcionales), elaboración del código fuente, pruebas y entrega del producto funcional, utilizando lenguajes orientados a objetos como Java, C o Python. *Fundamentos de IS, Cap. 2. Basado en Sommerville, I. (2011)*

3. ¿Qué tareas debemos realizar?

Entre las tareas principales se encuentran: definir requisitos, diseñar la arquitectura del sistema, codificar en un lenguaje de programación adecuado, ejecutar pruebas y validar el producto frente a los requisitos iniciales del cliente. *(Fundamentos de IS, Cap. 2. Basado en Pressman, R. S. (2005))*

4. ¿Cómo nos aseguramos de que haremos un gran trabajo?

Aplicando la calidad del software mediante métodos de ingeniería de software, prácticas de gestión sólidas y control de calidad integral, que incluyen estándares, revisiones entre pares y análisis estático del código. *(López Echeverry & Valencia Ayala (2008), citado en Fundamentos de IS, Cap. 2)*

5. ¿Cómo medimos la calidad de lo que hemos hecho?

A través de atributos de calidad del producto software basados en el modelo ISO/IEC 25010, que categoriza la calidad en características y subccaracterísticas como funcionalidad,

rendimiento, seguridad, mantenibilidad y fiabilidad. (ISO/IEC, 2011)*, citado en Fundamentos de IS, Cap. 2*

6. ¿Cómo nos aseguramos de que lo que hemos hecho esté realmente a la altura de las necesidades iniciales?

La calidad del software se logra cuando el sistema cumple los requisitos especificados y las necesidades del cliente. Se verifican tanto los requisitos funcionales como los no funcionales para garantizar que el producto entregado satisface las expectativas del usuario. *(López Echeverry & Valencia Ayala (2008), citado en Fundamentos de IS, Cap. 2)*

7. ¿Cómo minimizamos el esfuerzo para lograr la meta?

Reutilizando componentes de software existentes; la ingeniería orientada a la reutilización enfoca el proceso en la integración de componentes previamente desarrollados en el sistema, evitando construirlos desde cero y reduciendo esfuerzo y costos. *(Fundamentos de IS, Cap. 3. Basado en Sommerville, I. (2011))*

8. ¿Qué habilidades necesitamos?

Es recomendable iniciar aprendiendo C y luego abordar lenguajes de más alto nivel como Java o Python, cuya demanda es bastante alta. Quien conozca C estructurado encontrará facilidad para aprender otros lenguajes orientados a objetos. *(IngSistemas (2017), citado en Fundamentos de IS, Cap. 2)*

9. ¿Qué herramientas necesitamos?

Se requieren lenguajes de programación adecuados al contexto (C, Java, Python, Swift, entre otros), IDEs de desarrollo, herramientas de gestión de proyectos y control de

versiones, seleccionando cada herramienta según el tipo de sistema a construir. *(IngSistemas (2017), citado en Fundamentos de IS, Cap. 2)*

10. ¿Cuántas personas se necesitan?

Depende del tamaño y complejidad del proyecto. Los equipos deben dimensionarse en función de las fases del ciclo de vida y los roles necesarios, pudiendo ir desde equipos pequeños hasta estructuras más amplias para proyectos de gran escala. *(Fundamentos de IS, Cap. 2. Basado en Pressman, R. S. (2005))*

11. ¿Qué habilidades, perfiles, experiencia?

Se requieren roles como: analista de requisitos, programadores, diseñadores, testers e ingenieros de requisitos. Cada rol tiene responsabilidades específicas dentro del proceso de desarrollo, desde el análisis hasta la entrega del producto. *(Fundamentos de IS, Cap. 3. Basado en Sommerville, I. (2011))*

Autoevaluación del Capítulo 2 (Págs. 25–27)

1. ¿Qué es una metodología de desarrollo de software?

Es un conjunto de métodos que cubren todo el ciclo de vida de desarrollo de sistemas y que están unidos por un enfoque general filosófico. No se trata solo de lenguajes ni de rutinas, sino de una guía integral para el desarrollo. *(Fundamentos de IS, Cap. 2, pág. 25)*

2. Las actividades estructurales del proceso de desarrollo de software son:

Comunicación, planificación, modelado (análisis y diseño), construcción (código y prueba) y despliegue. Estas actividades estructurales constituyen el marco de referencia del proceso genérico de desarrollo de software. (Pressman, 2005). *Ingeniería del Software: Un Enfoque Práctico. McGraw-Hill*

3. Una de las razones por la cual se incrementan los costos al agregar funcionalidad después que un sistema está en operación es:

La práctica de desarrollo deficiente. Cuando no se aplican buenas prácticas de ingeniería desde el inicio, modificar o agregar funcionalidad en un sistema en operación implica un costo mucho mayor al que habría tenido durante el desarrollo original. *(Fundamentos de IS, Cap. 2, pág. 25)*

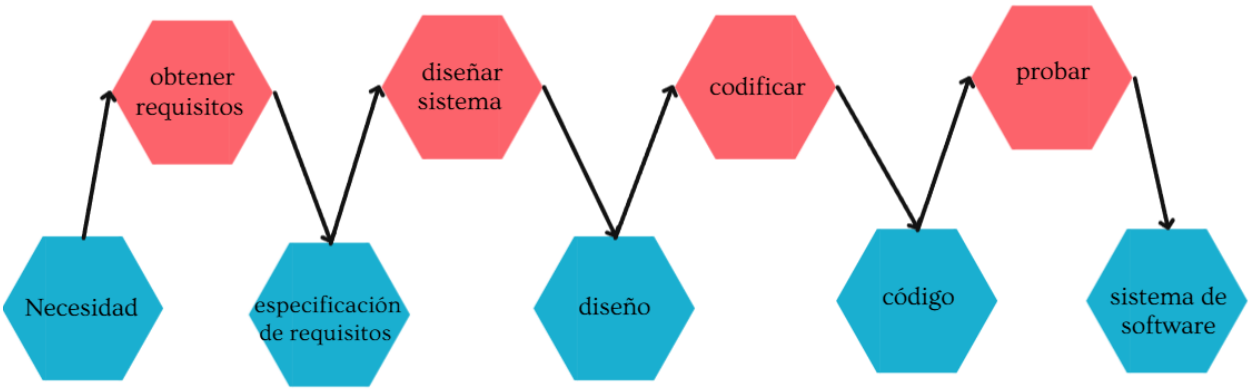
4. Ingeniería de software asistida por computador (CASE) significa:

Un conjunto de herramientas de software integradas para ser utilizadas por otras. Las herramientas CASE apoyan las actividades del proceso de desarrollo, como el análisis, diseño, codificación y pruebas. (Pressman, 2005). *Ingeniería del Software: Un Enfoque Práctico. McGraw-Hill*

5. La metodología con sus tipos correspondientes de clasificación correcta es:

La metodología XP (Extreme Programming) es de tipo orientada a objetos y ágil. Es incorrecta la clasificación de Scrum como 'estructurada' y del Proceso Unificado como 'estructurada', pues este último es iterativo e incremental. *(Fundamentos de IS, Cap. 2, pág. 26)*

6. En el gráfico propuesto, la secuencia ordenada de las figuras es:



Necesidad → obtener requisitos → especificación de requisitos → diseñar sistema → diseño → codificar → código → probar → sistema de software. Esta secuencia

representa el flujo típico del proceso de desarrollo de software. *(Fundamentos de IS, Cap. 2, pág. 26)*

7. Los modelos de calidad del producto software y los modelos de calidad del proceso de desarrollo son:

El modelo de calidad del producto software correcto es la norma ISO/IEC 25010. Esta norma identifica características y subccaracterísticas de calidad para evaluar el producto software de manera estandarizada. (ISO/IEC, 2011)*, citado en Fundamentos de IS, Cap. 2*

8. La norma ISO/IEC 25010 identifica características y subccaracterísticas de calidad. La opción correcta es:

La Eficiencia de desempeño es una característica (no subcaracterística) de calidad del producto software según ISO/IEC 25010. La seguridad y la fiabilidad también son características, no subccaracterísticas. (ISO/IEC, 2011)*, citado en Fundamentos de IS, Cap. 2)*

9. Según los roles y responsabilidades, ¿qué opción es correcta?

Mantener la configuración del código corresponde a un configuration manager (gestor de configuración). Este rol es responsable de controlar los cambios en el software y asegurar la integridad de los artefactos del proyecto. *(Fundamentos de IS, Cap. 2, pág. 27)*

10. Según los objetivos con la correspondiente fase del proceso unificado de desarrollo, ¿qué opción es correcta?

La fase de Inicio establece el alcance del proyecto y las condiciones de los límites, los criterios de aceptación, lo que se necesita en el producto y lo que no. Es la fase de definición inicial antes del análisis y diseño detallado. *(Fundamentos de IS, Cap. 2, pág. 27)*

CONCLUSIONES

- La ingeniería de software requiere priorizar metodologías estructuradas sobre la simple codificación para garantizar la escalabilidad técnica y la calidad del producto entregado.
- El modelo de calidad ISO/IEC 25010 proporciona un marco estandarizado de ocho características que permite evaluar de manera objetiva el grado en que el software satisface los requisitos del usuario.
- Los paradigmas iterativos e incrementales son más eficaces para el desarrollo actual, ya que absorben mejor los cambios en los requerimientos del cliente a lo largo del ciclo de vida.
- El rol de cada integrante del equipo (analista, programador, tester, configuration manager) debe estar claramente definido para garantizar la calidad y trazabilidad del proceso de desarrollo.

RECOMENDACIONES

- Aplicar desde el inicio buenas prácticas de ingeniería de software para evitar el incremento innecesario de costos al agregar funcionalidad en sistemas ya operativos.
- Implementar metodologías ágiles como XP o Scrum en proyectos con requisitos dinámicos, y metodologías plan-driven en sistemas críticos donde la seguridad es prioritaria.
- Definir métricas de calidad alineadas con la norma ISO/IEC 25010 y realizar validaciones con usuarios reales antes de liberar cualquier versión del sistema a producción.

BIBLIOGRAFÍA

ACM/IEEE. (2020). *Computing Curricula 2020 (CC2020)*. Obtenido de https://www.acm.org/binaries/content/assets/education/curricula-recommendations/cc2020.pdf

ISO/IEC. (2011). *Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models.* International Organization for Standardization.

Pressman, R. S. (2005). *Ingeniería del Software: Un Enfoque Práctico*. Obtenido de https://books.google.com/books?id=y4k_AQAAIAAJ

Ruiz Robalino, J. A. (2024). *Fundamentos de Ingeniería de Software, un Enfoque Práctico.* Universidad de las Fuerzas Armadas ESPE.

Sommerville, I. (2011). *Software Engineering*. Obtenido de https://web2.qatar.cmu.edu/cs/17313/Software-Engineering-9th-Edition-by-Ian-Sommerville.pdf

Integrantes
Jaime cabezas
Jelen Lucio
Bryan Cevallos